University of Massachusetts, Amherst
CICS
COMPSCI 683: Artificial Intelligence

Assignment 4: CSPs and Reinforcement Learning

A Markov Decision Process (MDP) is given by a set of *states* $\mathcal{S}$, a set of *actions* $\mathcal{A}$ and a *reward function* $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. When we take the action a at the state s, we *transition* to the next state according to the function $\delta : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$. Thus, $\delta(s, a) \in \mathcal{S}$ is the state we reach when we take the action a in the state s. The transition function δ can be probabilistic, in which case $\delta(s, a)$ is a probability distribution over $\mathcal{S}$.

Not all actions can be taken in all states. Given a state $s \in \mathcal{S}$, we let $A(s) \subseteq \mathcal{A}$ be the set of actions that can be taken in the state s. We say that a state is a *terminal state* if $A(s) = \emptyset$. In this case, one does not transition out of s, nor can one accrue additional reward.

We say that a policy π is *deterministic* if, given the current state s_t and the sequence of actions/states/rewards from time 0 to $t-1$, the action chosen next, $\pi(s_t)$, is not randomized. In other words, an adversary observing what the agent does at times $0, \ldots, t-1$ can completely predict what their action will be in time t. A policy π is randomized if we allow an agent to follow a distribution over actions, rather than a single action.

Given a deterministic policy π, let

$$V^{\pi}(s) = \sum_{t=0}^{\infty} \gamma^t r_{\pi}(s_t).$$

Here, $s_0 = s$, a_t is the action taken at time t according to the policy π, s_t is the state reached at time t, and $r_{\pi}(s_t)$ is the reward of the policy π at time t. More formally, $r_{\pi}(s) = r(s, \pi(s))$, where $\pi(s) \in A(s)$ is the action selected by the policy π.

If a policy is not deterministic, then we write $\pi(a \mid s)$ to be the probability of choosing the action $a \in A(s)$ at the state s according to the policy π. In this case the reward of π is simply

$$r_{\pi}(s) = \sum_{a \in A(s)} \pi(a \mid s) r(s, a).$$

The optimal policy is one that maximizes $V^{\pi}(s)$ over all policies π and over all states, i.e. for every state s we have that

$$V^*(s) = \max_{a \in A(s)} \{r(s, a) + \gamma V^*(\delta(s, a))\}.$$

1. **(15 points) Let $G = (V, E)$ be an *arbitrary* directed graph with weighted vertices. Every vertex $v \in V$ has a weight $w_v \geq 0$. A *prefix* of G is a subset of vertices $P \subseteq V$ such that there is no edge $u \to v$ where $u \notin P$ but $v \in P$. In other words, $P \subseteq V$ is a valid prefix if for every edge $u \to v \in E$, if $v \in P$ then $u \in P$. An instance of the Exact a-b Prefix problem consists of finding a prefix that has exactly a vertices and whose weight is exactly b, i.e., the number of vertices in P is a and the sum of the weights of vertices in P is exactly b. In what follows, you may only use binary variables of the form x_v where $x_v = 1$ if vertex $v \in V$ is part of the prefix, and 0 otherwise. Write the constraints describing a general instance of the Exact a-b Prefix problem. You may only use basic arithmetic symbols, standard logical operators ($\forall, \exists, \wedge, \vee, \neg$, etc), and standard set operators ($\in, \subseteq, \notin, \cap, \cup$, etc) when defining the constraints.**

Solution:

$$x_v \in \{0, 1\}, \forall v \in V$$

$$\sum_{v \in V} x_v = a$$

$$\sum_{v \in V} w_v x_v = b$$

$$x_v \leq x_u, \forall (u, v) \in E.$$

2. (15 points) Consider the CSP where there are four variables x, y, z, w with domain $D = \{1, 2, 3\}$ and they have the following constraints.

$$x + y \geq 2 \qquad x + w \leq 3$$
$$y + z \leq 4 \qquad z + w \geq 2$$
$$y + w \leq 5$$

(a) (3 points) Draw the constraint graph for this CSP

(b) (12 points) Before starting our backtracking search for a solution, we run the `AC3` Algorithm on this instance, with the initial edge queue below:

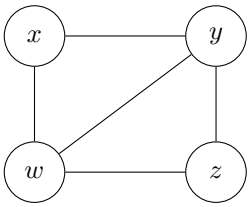
$$[(x, y), (x, w), (y, x), (y, z), (y, w), (z, y), (z, w), (w, x), (w, y), (w, z)]$$

Describe the run of the algorithm for this instance: state what domains are reduced (write `none` if unchanged) and what edges are added to the end of the queue in every iteration. You may write your answer by filling a table like this one.

Edge	Reduced domain	Edges added to queue
(x, y)		
(x, w)		
$\vdots$		

Solution:

a)



b)

Edge	Reduced domain	Edges added to queue
(x, y)	**none**	**none**
(x, w)	$D_x : \{1, 2, 3\} \to \{1, 2\}$	(y, x)
(y, x)	**none**	**none**
(y, z)	**none**	**none**
(y, w)	**none**	**none**
(z, y)	**none**	**none**
(z, w)	**none**	**none**
(w, x)	$D_w : \{1, 2, 3\} \to \{1, 2\}$	$(y, w), (z, w)$
(w, y)	**none**	**none**
(w, z)	**none**	**none**
(y, x)	**none**	**none**
(y, w)	**none**	**none**
(z, w)	**none**	**none**

3. (25 points) Consider the MDP graph: states are nodes, and there is a directed edge of the form (s, s') if there is some action a such that $\Pr[\delta(s, a) = s'] > 0$. In other words, there is an edge from s to s' if some action leads from s to s' with positive probability. Note that under this definition, terminal states do not have any outgoing edges.

Suppose that the MDP graph is acyclic. Present an algorithm for computing the optimal policy for acyclic MDPs. What is the runtime of your algorithm?

Solution:

1) Pseudocode:

Topologically sort the MDP graph.
for every state s do
    $V^*(s) \leftarrow 0$
end for

for states s in reverse topological order do
    if $A(s) = \emptyset$ then
        $V^*(s) \leftarrow 0$
    else

$$V^*(s) \leftarrow \max_{a \in A(s)} \left[r(s, a) + \gamma \sum_{s'} \Pr(s' \mid s, a) V^*(s') \right]$$

        $\pi^*(s) \leftarrow$ **an action a with max rewards**
    end if
end for

return V^* and π^*

2) Running Time:

$$\mathcal{O}(|S| + |E| + |A|) \qquad S: \ State, \ E: \ Edge, \ A: \ Action$$

4. (35 points) Tammy is training for a marathon, which takes place in three weeks. She needs to train for the marathon, however, she's recovering from an injury. Each week, she can either train or rest.

- **If she trains while injured, she gets a reward of 1.**
- **If she trains while fully recovered, she gets a reward of 2.**
- **If she rests, she gets a reward of 0.**

When Tammy trains while injured, she has a 50% chance of being fully recovered the following week. If she decides to rest instead, this probability increases to 80%. Suppose that once she recovers, she does not get injured again. Tammy does not earn any additional rewards after week 3, regardless of her being fully recovered or not.

(a) (10 points) Describe this problem as an MDP. Describe the states, actions at each state, and transition model. Indicate which states are terminal nodes, and the reward function in each state.
Hint: When modeling the problem, it is useful to have states representing Tammy being recovered or not on each week (except for week 1, she cannot be recovered then), and one terminal state for the marathon itself.

Solution:

1) State:

$$S = \{I_1, I_2, R_2, I_3, R_3, T\}$$

2) Action:

$$Non - Terminate : \{train, rest\}$$

$$Terminate : \{None\}$$

3) Transition Model:

- **From injured states**
  - **Tammy + injured + train:**

$$I_t \xrightarrow{\textbf{train}} \begin{cases} R_{t+1}, & 0.5, \\ I_{t+1}, & 0.5. \end{cases}$$

  - **Tammy + injured + rest:**

$$I_t \xrightarrow{\textbf{rest}} \begin{cases} R_{t+1}, & 0.8, \\ I_{t+1}, & 0.2. \end{cases}$$

- **From recovered states**

$$R_t \xrightarrow{\textbf{train}} R_{t+1}$$
$$R_t \xrightarrow{\textbf{rest}} R_{t+1}$$

- **Week 3 transitions**

$$I_3 \xrightarrow{\textbf{train}} T$$
$$I_3 \xrightarrow{\textbf{rest}} T$$
$$R_3 \xrightarrow{\textbf{train}} T$$
$$R_3 \xrightarrow{\textbf{rest}} T$$

4) Reward Model:

$$r(I_1, train) = r(I_2, train) = r(I_3, train) = 1$$
$$r(R_2, train) = r(R_3, train) = 2$$
$$r(rest) = 0$$

(b) (5 points) What are the possible (deterministic) policies are there for this MDP?

Solution:

$$2^5 = 32$$

(c) (10 points) Suppose Tammy decides to train every week. What is Tammy's value at every state for this policy as a function of the discount factor γ?

Solution:

$$V^\pi(T) = 0$$
$$V^\pi(I_3) = 1, \qquad V^\pi(R_3) = 2$$
$$V^\pi(I_2) = 1 + 1.5\gamma, \qquad V^\pi(R_2) = 2 + 2\gamma$$
$$V^\pi(I_1) = 1 + 1.5\gamma + 1.75\gamma^2$$

(d) (10 points) Compare the reward of the policy of training every week, to the policy of resting on the first week, and then training on weeks 2 and 3. For what values of the discount factor γ is one policy better than the other?

Solution:

$$V_A(I_1) = 1 + 1.5\gamma + 1.75\gamma^2$$
$$V_B(I_1) = 1.8\gamma + 1.9\gamma^2$$

1) Compare:

$$1 + 1.5\gamma + 1.75\gamma^2 > 1.8\gamma + 1.9\gamma^2$$
$$1 - 0.3\gamma - 0.15\gamma^2 > 0$$
$$1 - 0.3\gamma - 0.15\gamma^2 = 0$$
$$20 - 6\gamma - 3\gamma^2 = 0$$
$$3\gamma^2 + 6\gamma - 20 = 0$$
$$\gamma = \frac{-6 \pm \sqrt{36 + 240}}{6} = \frac{-6 \pm \sqrt{276}}{6} = \frac{-3 \pm \sqrt{69}}{3} = \frac{-3 + \sqrt{69}}{3}$$
$$\frac{-3 + \sqrt{69}}{3} \approx 1.77$$
$$V_A(I_1) > V_B(I_1), \quad 0 \leq \gamma \leq 1 < 1.77$$

5. **(10 points) Consider the following modification of the greedy algorithm for learning a regret-minimizing policy in the expert systems domain. Instead of picking an action out of the set of best-performing actions so far (as is the case for the simple greedy algorithm), we pick the best performing action in the previous k time-steps; if there is more than one k-best performing action, we pick the one with the lowest index. For example, if $k = 1$, we simply pick any one of the actions that offered the highest reward in the previous round; if $k = 3$, we pick the action that had the highest cumulative reward in the last three rounds, etc. More formally, the k-greedy algorithm works as follows. At time t, let S_t be the set of actions that had maximal total reward at time steps $t - k, ..., t - 1$; we pick the action with the lowest index out of S_t (if $t \leq k$ we run the original greedy algorithm). What is the worst-case regret of the k-greedy algorithm? You must provide a formal proof of its regret guarantees with respect to the best action.**

Solution:

$$R_T = V_{\text{best}}(T) - V_{\text{alg}}(T), \qquad V_{\text{best}}(T) = \max_i \sum_{t=1}^{T} r_t(i), \qquad V_{\text{alg}}(T) = \sum_{t=1}^{T} r_t(A_t).$$

Assume rewards are in $\{0, 1\}$.

1) Upper Bound:

$$R_T \leq T,$$

2) Lower Bound, let's consider two actions a_1, a_2.

$$r_t(A_t) = 0, \qquad A_t \text{ is the action alg choose}$$
$$r_t(a) = 1. \qquad a \neq A_t$$

Then

$$V_{\text{alg}}(T) = \sum_{t=1}^{T} r_t(A_t) = 0.$$

Also, in every round,

$$r_t(a_1) + r_t(a_2) = 1.$$

Hence

$$\sum_{t=1}^{T} r_t(a_1) + \sum_{t=1}^{T} r_t(a_2) = T.$$

Therefore,

$$V_{\text{best}}(T) = \max\left\{\sum_{t=1}^{T} r_t(a_1), \sum_{t=1}^{T} r_t(a_2)\right\} \geq \frac{T}{2}.$$

Thus,

$$R_T = V_{\text{best}}(T) - V_{\text{alg}}(T) \geq \frac{T}{2} - 0 = \frac{T}{2}.$$

$$R_T = \Theta(T)$$

6. **(bonus points) Consider the no regret learning setting we have studied in class. We saw that in order to achieve a low regret, we need a sufficiently large number of rounds. What happens when the number of rounds is small? Prove that if there are n experts and T time steps, such that $T < \lfloor \log_2 n \rfloor$, then for any randomized algorithm $\mathcal{A}$, there is some randomized assignment of losses[1] in $\{0, 1\}$ to the experts (which can depend on what $\mathcal{A}$ does) such that**

   1. **$\mathcal{A}$ incurs a loss of at least $\frac{T}{2}$.**
   2. **There exists at least one action that has a loss of 0.**

   You may assume (with not much loss of generality) that $n = 2^k$ for some positive integer k.

Solution:

$$n = 2^k, \qquad T < \lfloor \log_2 n \rfloor$$

$$S_0 = \{1, 2, \ldots, n\}$$

1)

At round t, let $p_t(i)$ be the probability that algorithm A chooses expert i.

Split S_{t-1} into two equal parts:

$$|S_t^0| = |S_t^1| = \frac{|S_{t-1}|}{2}.$$

Define

$$P_0 = \sum_{i \in S_t^0} p_t(i), \qquad P_1 = \sum_{i \in S_t^1} p_t(i).$$

The adversary assigns loss 1 to the half with larger probability, and loss 0 to the other half.

Thus,

$$\mathbb{E}[\ell_t(A)] \geq \max(P_0, P_1) \geq \frac{1}{2}.$$

Therefore,

$$\mathbb{E}[L_A] = \sum_{t=1}^{T} \mathbb{E}[\ell_t(A)] \geq \sum_{t=1}^{T} \frac{1}{2} = \frac{T}{2}.$$

2)

After T rounds,

[1] Here we consider losses instead of rewards, but the idea is similar.

$$|S_T| = \frac{n}{2^T} = \frac{2^k}{2^T} = 2^{k-T}.$$

Since $T < k$**,**

$$|S_T| = 2^{k-T} \geq 1.$$

Hence, there exists an expert $i \in S_T$ **such that**

$$L_i = 0.$$